TP9 – Gestion des exceptions, erreurs et contrôle des champs

Développement Mobile Android (Java)

BTS – Développement Informatique

Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :
- Contrôler les champs d'un formulaire Android
- Gérer les erreurs de saisie utilisateur
- Utiliser les blocs `try / catch`
- Éviter les plantages de l'application
- Afficher des messages d'erreur clairs

Contexte

On souhaite développer une application Android simple contenant un formulaire d'inscription composé de :
- Nom
- Âge
- Email
- Bouton **Valider**

L'application doit vérifier la validité des données avant validation, et les données doivent être stockées dans une base de données.

Par la suite, l'utilisateur devra pouvoir se connecter, si les informations qu'ils va entrer sont conformes aux données stockées dans la base de données, l'utilisateur pourra accéder à l'application, sinon un message d'erreur devra s'afficher jusqu'à ce qu'il entre les données correctes.

Travail demandé

Vous devez développer une application Android complète respectant les contraintes suivantes.

1. Interface à réaliser

L'application doit contenir :
- Une fenêtre d'acceuil avec deux boutons : login et signUp.
- Une fenêtre d'inscription contenant :
  - Un champ **Nom**

    - Un champ **Âge**
    - Un champ **Email**
    - Un champ **Mot de passe**
    - Un bouton **Valider**
  - Une fenêtre de login.
  - Une fenêtre de Dashboard, quand l'utilisateur va accéder à son compte.

2. Comportement attendu

L'application doit :

- Refuser les champs vides
- Refuser un âge non numérique
- Refuser un âge négatif ou supérieur à 120
- Vérifier que l'email contient le caractère `@`
- Vérifier que le mot de passe contient au moins 6 caractères
- Utiliser un bloc `try / catch`
- Empêcher le crash de l'application
- Désactiver le bouton tant que le formulaire est invalide

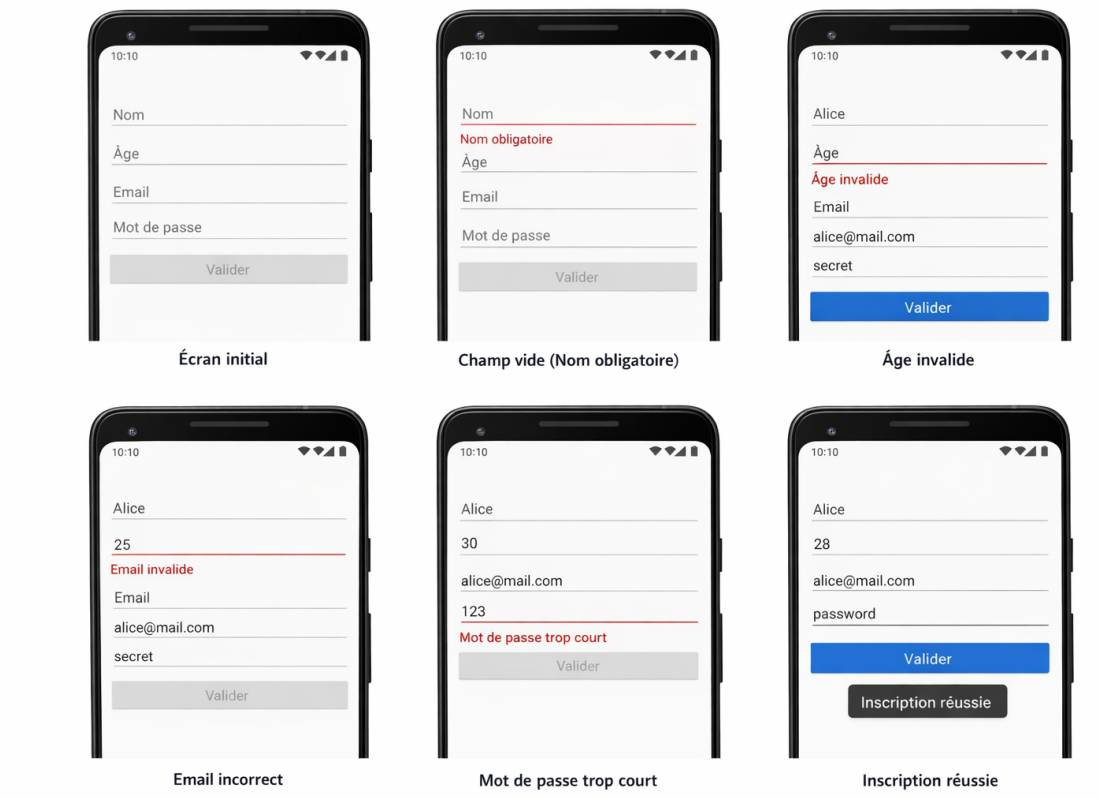

FIGURE 1 – Validation dynamique du formulaire : écran initial, champ vide, âge invalide, email incorrect, mot de passe trop court et inscription réussie.

3. Mise en place de SQLite et liaison avec l'application

Organisation recommandée du projet :

```
com.example.nomduprojet
 activities
 database
 models
 adapters (si besoin)
 utils
```

Création des tables

Table Utilisateur

Champ	Type
id	INTEGER PRIMARY KEY AUTOINCREMENT
age	INTEGER
nom	TEXT
email	TEXT
password	TEXT

Vous pouvez ajouter d'autres tables si vous en avez besoin.

Captures d'écran obligatoires

Vous devez insérer dans votre rapport les captures suivantes :

1. Interface d'acceuil
2. Interface d'inscription :
   — Écran initial (bouton désactivé)
   — Erreur affichée si champ vide
   — Erreur si âge invalide
   — Erreur si email incorrect
   — Erreur si mot de passe trop court
   — Capture après validation réussie (Toast affiché)
3. Interface de login avec erreur quand le mot de passe est incorrecte
4. Interface de tableau de bord

Chaque capture doit être accompagnée d'un court commentaire explicatif.

Code à fournir

Dans votre rapport PDF, vous devez insérer :
— Le fichier `activity_main.xml`
— Le fichier `MainActivity.java`
— Le fichier `LoginActivity.java`
— Le fichier `activity_login.xml`
— Le fichier `SignUpActivity.java`
— Le fichier `activity_signup.xml`
— Le fichier `DashboardActivity.java`
— Le fichier `activity_dashboard.xml`
— La méthode `validerFormulaire()`

- La méthode `verifierFormulaire()`
- L'implémentation du `TextWatcher`
- Code Database et DAO

Le code doit être bien indenté et commenté.

Questions de réflexion

Répondez brièvement aux questions suivantes :

1. Pourquoi utilise-t-on `try / catch` ?
2. Quelle est la différence entre `setError()` et `Toast` ?
3. Pourquoi désactiver le bouton avant validation ?
4. Que se passe-t-il si on supprime la gestion des exceptions ?

Livrables attendus

- Projet Android publié sur Github.
- Rapport PDF contenant :
  - Les captures
  - Le code
  - Les réponses aux questions